LEARN BEING FORWARD

Programming — C++ Programming

Templates & generic programming

COMPLETE LEARNING GUIDE

What You'll Learn

Templates are how C++ achieves generic programming — writing a single function or class that works correctly across many different types, with the compiler generating a specialized version for each type actually used, rather than relying on runtime type erasure the way some other languages' generics do. This compile-time instantiation model is both templates' greatest strength (zero runtime overhead, full type safety, aggressive optimization opportunity for the compiler) and their most common source of confusion for newcomers, especially around where template code must physically live and the sometimes-cryptic errors that come from an incompatible type being substituted in. This sub-module covers function and class templates, how type deduction works, template specialization for special-casing specific types, variadic templates for accepting an arbitrary number of arguments, constexpr for compile-time computation, and a brief, practical look at C++20 concepts for constraining what types a template accepts.

1. Function Templates

A function template (template <typename T> T myMax(T a, T b) { ... }) defines a blueprint the compiler uses to generate an actual, concrete function for each distinct type T it's called with — calling myMax(3, 5) instantiates an int version, and myMax(3.5, 2.1) instantiates a separate double version, each compiled and type-checked independently as if you'd written them by hand. This is fundamentally different from a language with type erasure: there's no single 'generic' myMax function at runtime, only however many concrete instantiations the compiler actually generated based on real call sites in your program, which is why a template that's never called with a given type never generates (or needs to compile) code for that type.

```cpp
#include <iostream>

template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << myMax(3, 5) << "\n";       // instantiates myMax<int>
    std::cout << myMax(3.5, 2.1) << "\n";   // instantiates myMax<double>
    std::cout << myMax(std::string("apple"), std::string("banana")) << "\n"; // myMax<std::string>
    return 0;
}
```

2. Class Templates

A class template parameterizes an entire class on one or more types, letting a single Stack<T> definition serve as a stack of ints, a stack of strings, or a stack of any other type — std::vector<T> itself is exactly this pattern, a class template the standard library ships rather than a built-in language feature. Member functions of a class template are themselves templates in disguise and, like function templates, are only compiled for the specific T's the class is actually instantiated with in your program, which is why class template definitions are almost always written entirely in header files rather than split into separate .cpp files the way ordinary classes are.

```cpp
#include <vector>
#include <iostream>

template <typename T>
class Stack {
public:
    void push(const T &value) { data_.push_back(value); }
    void pop() { data_.pop_back(); }
    const T &top() const { return data_.back(); }
    bool empty() const { return data_.empty(); }
private:
    std::vector<T> data_;
};

int main() {
    Stack<int> intStack;
    intStack.push(1);
    intStack.push(2);
    std::cout << intStack.top() << "\n"; // 2

    Stack<std::string> stringStack;
    stringStack.push("hello");
    std::cout << stringStack.top() << "\n"; // hello
    return 0;
}
```

3. Template Type Deduction

When calling a function template, the compiler usually infers (deduces) the template type parameter directly from the arguments supplied, so myMax(3, 5) doesn't need to be written as myMax<int>(3, 5) — the explicit form is only needed when the types can't be deduced unambiguously, such as when the two arguments have genuinely different types and neither should implicitly convert to the other. auto, introduced for ordinary variables, uses a closely related deduction mechanism: auto x = myMax(3, 5); infers x's type from the function's return value, which is especially useful when that return type is long or itself template-dependent.

- myMax(3, 5.0) is a compile error (or forces an explicit myMax<double>(3, 5.0)) because the two arguments are different types and T can't be deduced as both int and double simultaneously.
- auto combined with a templated return type avoids having to spell out complex, nested template type names explicitly.

4. Template Specialization

Full specialization lets you provide a completely different implementation for one specific type, overriding the generic template entirely for that case — useful when the generic logic genuinely doesn't make sense or isn't optimal for a particular type (comparing C-style char* strings with > doesn't compare their contents the way a generic myMax expects, so a specialization for char* using strcmp would fix that specific case). Partial specialization (available for class templates, not function templates) specializes for a pattern of types rather than one exact type, such as specializing a container template specifically for pointer types to add pointer-specific behavior.

```cpp
#include <cstring>
#include <iostream>

template <typename T>
T myMax(T a, T b) { return (a > b) ? a : b; }

// Full specialization for const char* — compares string CONTENTS, not pointer addresses
template <>
const char *myMax<const char *>(const char *a, const char *b) {
    return (std::strcmp(a, b) > 0) ? a : b;
}

int main() {
    std::cout << myMax(3, 7) << "\n";                 // 7 — generic version
    std::cout << myMax("apple", "banana") << "\n";    // banana — specialized version
    return 0;
}
```

5. Variadic Templates

A variadic template (template <typename... Args>) accepts an arbitrary, unspecified number of template arguments, which is how functions like printf-replacement logging utilities or std::make_unique can accept any number of constructor arguments of any types. Processing a parameter pack typically uses recursion (peel off one argument, recurse on the rest, with a base case for zero or one remaining argument) in pre-C++17 style, or the C++17 fold expression (args + ...) for simple cases that reduce to one operator applied across the whole pack.

```cpp
#include <iostream>

// Base case: no arguments left
void print() { std::cout << "\n"; }

// Recursive case: peel off one argument, recurse on the rest
template <typename T, typename... Rest>
void print(T first, Rest... rest) {
    std::cout << first << " ";
    print(rest...); // recurse with one fewer argument
}

int main() {
    print(1, "two", 3.0, '4'); // 1 two 3 4
    return 0;
}
```

6. constexpr and Compile-Time Computation

constexpr marks a function or variable as eligible to be evaluated entirely at compile time when its inputs are themselves compile-time constants, producing the result as part of the compiled binary with zero runtime cost — a constexpr factorial(5) can become the literal value 120 baked directly into the program, with no function call happening at runtime at all. This differs from const, which only promises a value won't change after initialization but doesn't require (or guarantee) compile-time evaluation; constexpr additionally interacts naturally with templates, since template arguments (like array sizes) must themselves be compile-time constants.

```cpp
#include <iostream>

constexpr int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

int main() {
    constexpr int result = factorial(5); // computed entirely at COMPILE time
    int arr[result];                     // valid — result is a compile-time constant
    std::cout << result << "\n";         // 120
    return 0;
}
```

7. A Glimpse of Concepts (C++20)

Before C++20, a template parameter with no constraints accepts ANY type, and a type that's incompatible with the template's body only produces an error deep inside the instantiated code — often a long, hard-to-parse error message pointing at internal template machinery rather than the actual mistake. Concepts let you constrain a template parameter to types satisfying specific requirements (std::integral, std::copyable, or a custom-defined concept), so an incompatible type produces a clear, immediate error at the call site instead — concepts are a readability and error-message improvement over older techniques like SFINAE and std::enable_if, without changing what templates can fundamentally do.

```cpp
#include <concepts>
#include <iostream>

template <std::integral T> // constrains T to integer types only
T doubleIt(T value) {
    return value * 2;
}

int main() {
    std::cout << doubleIt(21) << "\n"; // 42 — int satisfies std::integral
    // doubleIt(3.14); // would fail to COMPILE with a clear concept-violation message
    return 0;
}
```

Common Pitfalls

- Splitting a class template's member function definitions into a separate .cpp file the way ordinary classes do, causing linker errors (template definitions generally must stay visible in the header).
- Calling a function template with two arguments of genuinely different types and expecting automatic deduction to just 'figure it out' by converting one.

- Writing a variadic template's recursive case without a matching base case, causing infinite template recursion that explodes compile time or fails outright.
- Assuming templates behave like runtime generics with type erasure, when in fact each distinct type instantiates entirely separate compiled code (template bloat).
- Confusing const (no compile-time evaluation guarantee) with constexpr (eligible for compile-time evaluation) when a genuinely compile-time constant is required, such as an array size.
- Forgetting typename before a dependent type name inside a template body, which the compiler can't otherwise parse as a type.

Key Takeaways

- Templates generate a separate, fully-compiled version of a function/class for each distinct type actually used — there's no runtime type erasure.
- Class (and function) template definitions must generally stay visible in the header, not split into a separate .cpp file.
- Full specialization overrides a template's behavior entirely for one specific type; partial specialization (classes only) targets a pattern of types.
- Variadic templates process an arbitrary number of arguments via recursion (with a required base case) or C++17 fold expressions.
- constexpr enables genuine compile-time computation, unlike const, which only guarantees immutability after initialization.
- C++20 concepts constrain template parameters to types meeting specific requirements, turning deep, cryptic instantiation errors into clear call-site errors.

Learn More

cppreference — Templates
https://en.cppreference.com/w/cpp/language/templates

cppreference — Constraints and concepts
https://en.cppreference.com/w/cpp/language/constraints